Chapter - 31

Advanced Java / Java EE

- Java EE stands for Java Enterprise Edition
- Java EE is used to develop web applications and enterprise applications .

Components of Java EE :-

- Web Components
    - used tp develop web applications
    - servlets
    -JSP (Java Server Pages)
- Enterprise Components
    - Used to develop enterprise application (secured, distributed, transaction management)
    - EJB (Enterprise Java Beans)

Java EE applications are used by multiple clients(users) and in order to make the Java EE applications to be used by multiple clients we need to deploy(copy) the application onto the server.

Types of Servers :-

- Web Servers
    -used to develop web applications only
        - (the server can understand only) use servlets and JSP

  Ex :- Servers are :-
    - Apache Tomcat
    - Adobe JRun
    - Jetty

- Application Servers
    - used to develop web application and enterprise applications
    - Servlets, JSP and EJB

    Ex: - servers are :-
        - Oracle GlassFish
        -Oracle WebLogic
        - IBM WebSphere
        - RedHat JBOSS

Servlets :-

- Servlets are Java programs which are used to develop server side dynamic web applications .

Which Technology are used to develop server side dynamic web applications :-
---

- - Servlets
  - JSP
  - ASP.NET
  - PHP
  - Django (using Python)

Servlets Hierarchy :-

- In servlets Hierarchy first topmost Interface is (Servlet interface).
- In Interface contain abstract methods (bydefault) .
- The method present in servlet interface are :-
  - init()
  - service()
  -destroy()
  -getServletInfo()
  -getServletConfig()
  -getServletContext()

2 – servlet interface have subclass which is (GenericServlet class)

- GenericServlet class implements the serlet interface.
- all method of servlet interface are overridden, except service().
- service() is abstract .
- If we are not overridden all class of interface then we have to make our class also abstract.

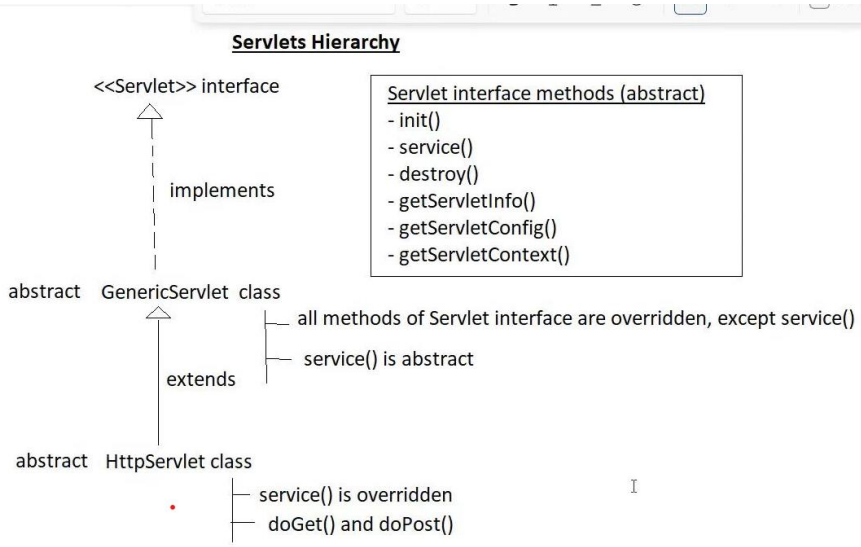
3 – GenericServlet class have subclass which is (HTTPServlet class).

- HTTPServlet class extends the GenericServlet class
- we have to overridden service() method in HTTPServlet class which is remaing abstract in upper class. (because service() is abstract).
- we can't create objects for abstract classes we can create only references.

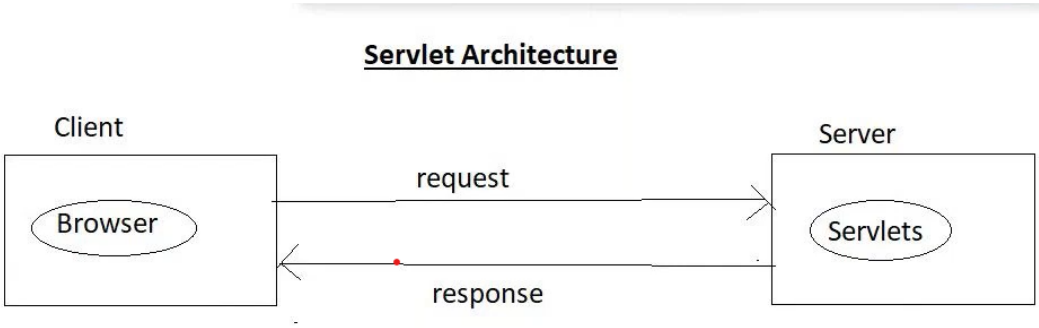
Note - In java form a rule that :- whenever a programmer want to create a servlets . they can't use generic servlet class or HTTPServlet class directly . we can't create objects of it . to form a rule that not to create objects for this classes they made this one as abstract.

- once we make abstract we have to compulsory to create subclass .
- When we go with HTTP Servlet we generally overridden two methods doGet() and doPost() methods.

Refer diagram Servlets Hierarchy.png



Servlet Architecture :-
- The data which goes from browsers to server is called Requests .
- The data which is coming from server to browsers is called Response.



Servlet Architecture.png

Servlet Life Cycle :-

Life Cycle is a collection of methods which are executed in an order automatically by the servlet container (server).

The three life cycle method are :-

- init()
  - called only once when servlet is loaded.
  - generally we establish database connection.

-service()
  - called for every requests (Ex:- login krna , payment krna every requests)

-destroy() - called only once when servlet is destroyed.
  - generally we close database connection.

Creating a Servlet :-

- By Creating a class which extends GenericServlet class .
- OR
- By Creating a class which extends HttpServlet class

Using GenericServlet class :- // In GenericServlet class we have to overridden service() compulsory.

```
public class WelcomeServlet extends GenericServlet
{
        public void service(ServletRequest request, ServletResponse response)
                throws ServletException, IOException
        {
                //handle the request
        }
 }
```

Using HttpServlet class :-

Note :-In HttpServlet don't have any abstract method that generally when we go with HTTP servlet we overridden two methods doGet() and doPost() methods.

```
public class WelcomeServlet extends HttpServlet
{
        public void doGet(HttpServletRequest request, HttpServletResponse response)
                throws ServletException, IOException
        {
                //handle the request
        }

        public void doPost(HttpServletRequest request, HttpServletResponse response)
                throws ServletException, IOException
        {
                //handle the request
        }
 }
```

Note :-

-interface HttpServletRequest extends ServletRequest{}

- interface HttpServletResponse extends ServletResponse{}

Note :-

class to class => extends

class to interface => implements

interface to interface => extends

interface to class => error

GenericServlet vs HttpServlet :-

- GenericServlet is used for different protocols like ftp, smtp, http etc.
- where as HttpServlet is used for http protocol only

ftp = file transfer protocol
smtp = simple mail transfer protocol
http = hypertext transfer protocol

Chapter - 32

Download and Install Apache Tomcat Server v9 :-

- Google : tomcat 9 download

Creating a Servlet in Eclipse using Apache Tomcat Server v9 :-
---
- Configure tomcat server in eclipse workspace

click on windows -> preferences -> servers -> Runtime Environments -> click on Add -> Select Apache -> Apache Tomcat Server v9.0 -> click Next -> clikc on browse -> select tomcat installation directory (tomcat v9) -> click select folder -> click Finish -> click Apply and Close .

- Creating a Dynammic Web Project :-

Click on file - > New -> dynamic web project
OR
Click on file - > New -> Other -> Web -> dynamic web project

Project Name : WebProj
Dynammic Web Module Version : 2.5 (Select)
Click finish

- Create a Servlet in WebProj :-

Right click on WebProj -> New -> Servlet
classname : WelcomeServlet
and click Finish

Refer program WelcomeServlet.java

- Runs the Servlet :-

Right click on WelcomeServlet.java -> Run as -> Run on Server

select Apache -> Apache Tomcat Server v9 -> click finish

which start the tomcat server and opens a browser with the following client url

http://localhost:8080/WebProj/WelcomeServlet

and display "welcome to Servlets onto the browsers .

Chapter - 33

http://localhost:8080/WebProj/WelcomeServlet

Note :- WelcomeServlet - It is not class . It is url-pattern

http => hypertext transfer protocol
localhost => IP Address or Host name of tomcat server
8080 => port number where tomcat server is running
WebProj => Project name / Application Name
WelcomServlet => url pattern

- when a request is made to a web application, the server loads web.xml
- web.xml file is the deployment descriptor / configuration file required for web applications
- In a web application we can have only one web.xml file.
- <web-app> is the root element of web.xml file.
-web.xml file is present in WEB-INF folder (WebProj/src/main/webapp folder)

Note :- WelcomeServlet - It is not class . It is url-pattern

Ex : **http://localhost:8080/WebProj/WelcomeServlet**

web.xml :-

```
<web-app>
	<servlet>
		<servlet-name>WelcomeServlet</servlet-name>
		<servlet-class>WelcomeServlet</servlet-class>
	</servlet>
	<servlet-mapping>
		<servlet-name>WelcomeServlet</servlet-name>
		<url-pattern>WelcomeServlet</url-pattern>
	<servlet-mapping>
</web-app>
```

Based on below web.xml file , the url to access WelcomeServlet is

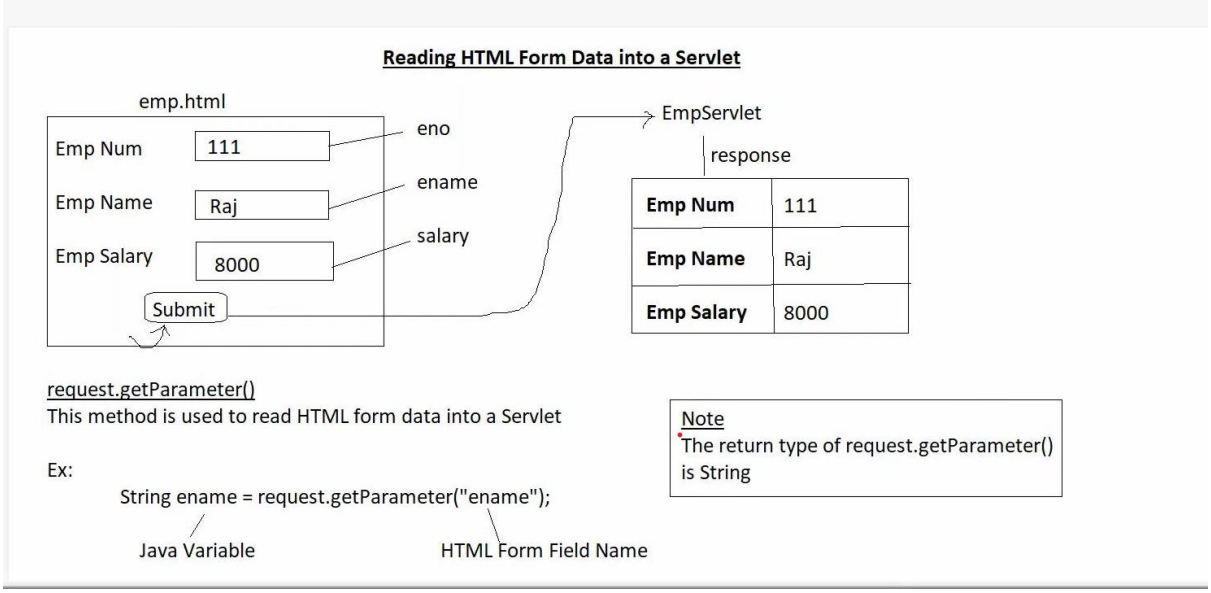
http://localhost:8080/WebProj/Wel

web.xml :-

```
<web-app>
        <servlet>
                <servlet-name>Welcome</servlet-name>
                <servlet-class>WelcomeServlet</servlet-class>
        </servlet>
        <servlet-mapping>
                <servlet-name> Welcome </servlet-name>
                <url-pattern>Wel</url-pattern>
        <servlet-mapping>
</web-app>
```

Reading HTML Form data into a Servlet :-

HTML Form is used to accept the user input .



Refer diagram Read HTML form Data.png

Example Application :-

- Creating a HTML file “emp.html” in WebProj
  Right Click on WebProj -> New -> HTML file
  Filename : emp.html
  and click Finish
  Refer program emp.html

- Create a Servlet “EmpServlet” in WebProj
  Refer program EmpServlet.java
- Run the application emp.html

Note :- So the suggested programming is always override doGet() & doPost() method (in servlet) and doGet() method handle the request and dopost() method invoke doGet() method. So that your servlet code in independent if method call because HTML program is wriiten by some other programmer and servlet code is written by other programmer . if HTML Programmer uses get() and however code is present in doGet()

. if HTML Programmer uses post() and however you are invoking doGet() from doPost() so So the suggested programming is always override doGet() & doPost() method (in servlet)
and handle the request in doGet() method and invoke doGet() method invoke from doPost() method.

GET vs POST

- In case of GET method, the data goes along with the url (visible in url) where as
 In case of POST method, the data goes as a separate page (not visible in url).

Note – When form contain passwords make use of post method

Note :- (ServletConfig interface & ServletContext interface we don’t use in real life but it is important for interview)
ServletConfig interface :-

- This interface is used to read initial parameters from web.xml file into a particular servlet.
  (from web.xml file also we can send some data to servlet that data we call initial parameters so read that data we use ServletConfig interface)
  Refer Programs
    - ServletConifDemo.java (servlet)
    - web.xml (we pass parameters from web.xml)

Ex:- (how to pass parameters in web.xml)

```
  <description></description>
   <display-name>ServletConfigDemo</display-name>
   <servlet-name>ServletConfigDemo</servlet-name>
   <servlet-class>ServletConfigDemo</servlet-class>
   <init-param>
   <param-name>username</param-name>
   <param-value>admin</param-value>
   </init-param>
   <init-param>
   <param-name>password</param-name>
   <param-value>root</param-value>
   </init-param>
 </servlet>
```

```
  <servlet-mapping>
    <servlet-name>ServletConfigDemo</servlet-name>
    <url-pattern>/ServletConfigDemo</url-pattern>
  </servlet-mapping>
</web-app>
```

ServletContext interface :-

- This interface is used to read context parameters (global) from wb.xml file into all services of web applications .

Refer Programs

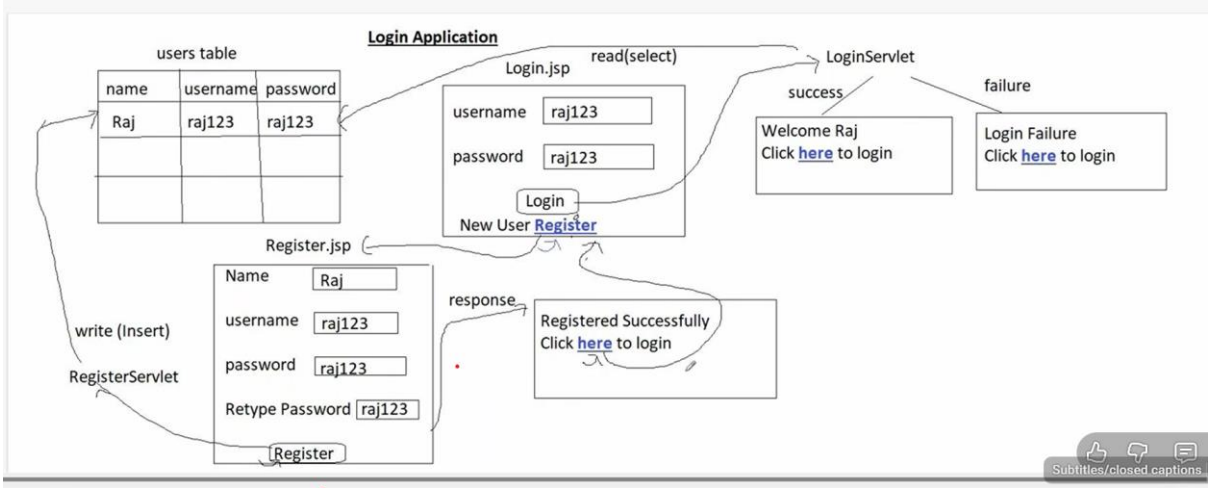
- ServletContextDemo.java (servlet)
- web.xml

```
Ex:- (we have to give value in webapp )

<web-app>
 <context-param>
  <param-name>goldrate</param-name>
  <param-value>50000</param-value>
  </context-param>
  <servlet>
    <description></description>
    <display-name>ServletContextDemo</display-name>
    <servlet-name>ServletContextDemo</servlet-name>
    <servlet-class>ServletContextDemo</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ServletContextDemo</servlet-name>
    <url-pattern>/ServletContextDemo</url-pattern>
  </servlet-mapping>
</web-app>
```

Chapter - 34

Login Application :-



Refer Diagram Login Application.png

To Create Login Application :-

- Create table users in MySQL

1 => mysql>create table users (name varchar(10), username varchar(10), password varchar(10));

- 2 => Create a Welcome page "Login.jsp" in webProj
  Refer program Login.jsp
  3 => Create a JSP file "Register.jsp" in webProj
  Refer Program Register.jsp

4 => Create a Servlet " RegisterServlet" in WebProj
Refer Program RegisterServlet.java

5 => Create a Servlet "LoginServlet" in WebProj
Refer Program LoginServlet.java

6 => Run the application

Setting for Login Application :-

- copy "mysql-connector-java-8.0.30.jar" into (WEB-INF/ lib) library folder .

Chapter - 35

JSP :-

- JSP stands for Java Server Pages .
- JSP is used to develop server side dynamic web applications like servlets.

Servlets vs JSP :-

- In Case of Servlets , Java Code contain HTML code Where as in case of JSP, HTMl code contains Java code.
- In case of Servlets, if the code is modified we need to recompile and redeploy the application (to make changes) where as in case of JSP, if the code is modified we don't need to recompile and redeploy the application .
- JSP code is less than Servlet as (in JSP we can use JSP tags, JSP Implicit Objects and JSTL.

JSP Life Cycle :-

1- jspInit()
    - Called only once when JSP page is loaded.
2- _jspService()
    - called for Every Request
    - Can't be Overriden
3- jspDestroy()
    - Called only once when JSP page is destroyed .

JSP Tags :-
---
1 – Directive tags
    - page directive tag => <%@page . . . . %>
    -include directive tag => <%@include . . . . %>
    - taglib directive tag => <%@taglib . . . . %>
2 - Scriptlet tags :-
    - Declaration tag = > <%! . . . . %>
    - Expression tag => <%= . . . . %>
    -Script tag => <% . . . . %>
3- Action tags :-
    - <jsp:include>
    - <jsp:forward>
    - <jsp:param>
    - <jsp:useBean>
    - <jsp:setProperty>
    - <jsp:getProperty>
4- Custom tags (self Learning)
    - used to create user defined tags in JSP .

Directive Tags :-

Page Directive Tag :-

- <%@page . . . %>
- It is used to import java packages .

Ex:-

<%@page import="java.util.*" %>

<%@page import="java.sql.*" %>

OR

<%@page import="java.util.*, java.sql.*" %>

Refer Program Date.jsp

Note :- In jsp we can import multiple packages in one time

include directive tag :-

- <%@include . . . %>
- used to include other JSP files .

Ex:-

<%@ include file="header.jsp" %>

. . . . .

<%@include file="footer.jsp" %>

Refer programs

-header.jsp

-footer.jsp

-includedemo.jsp (run)

(In JSP three people can work easily then we can add those file) The division of work will be easy in jsp.

Scriplet Tags :-

Declaration tags :-

- <%! . . . . %>
- It is used to declare variables and to define methods .

Ex:

<%! int n = 100; %>

```
<%! Void xxx()
        {
        . . . .
        }
        %>

<%! Void jspInit() {
```

```
        . . . . .
            }
```

Expression tag :-

- <%= . . . %>
- used to acess the value of the variable and to call methods
- the code goes into _jspService()
- Ex :-

```
<%= n %>    // 100 is displayed on browser
<%= XXX() %>
<%= new java.util.Date() %>
```

Script tag :-

- <% . . . %>
- used to include Java Statements
- use code goes into _jspservice()

Ex :-

```
<%
for(int i = 1; i<=10; i++ )
out.println(i);
%>
```

- out is an implicit object of JSP and it is an object of class JspWriter.
- JspWriter is a subclass of PrintWriter

Q i- Create a HTML file which h accepts a number from users .

ii– Create a JSP file which reads the number from HTML file and displays the factorial of the given number ?

Refer Programs

- fact .html (run)
- Factorial.js

Q i – Create a HTML file which accepts a color from user.

ii - Create a JSP file which reads the color from HTML file and sets the background of webpage with the given color .

Refer diagram BGColor.png

Note :- request is an implicit object of JSP which is an object of HttpServletRequest interface .

Chapter - 36

JSP Tags :-
---
1 - **page directive tag** => <%@page . . . %>
It is used to import java packages .
2 – **include directive tag** => <%@include . . . %>
It is used to include other JSP files .
3 – **Declaration tag** => <%! . . . %>
It is used to declare variables and to define methods .
4 - **Expression tag** => <%= . . . %>
It is used to access the value of variable and to call methods .
5 – **Script tag** => <% . . . %>
It is used to include Java Statements

JSP Action tags :-

- <jsp:include>
- <jsp:forward>
- <jsp:param>
- <jsp:useBean>
- <jsp:setProperty>
- < jsp:getProperty>

1 or 2 => <jsp:include> and <jsp:forward>
used to include other JSP files
Refer programs
- First.jsp (run)
- Second.jsp
-Third.jsp

Note :- Whenever you want to handle the response by all pages then go with include
If you don't want to come back to previous pages go with forward

In case of <jsp:include> the response is handled by all the JSP pages Where as in case of <jsp:forward> the response is handles by only the last page

3 - <jsp:param> :-

Used to send the parameters from one JSP page to another JSP page
ReferPrograms
- JspParam1.jsp (run)
- JspParam2.jsp

Using JavaBeans in JSP

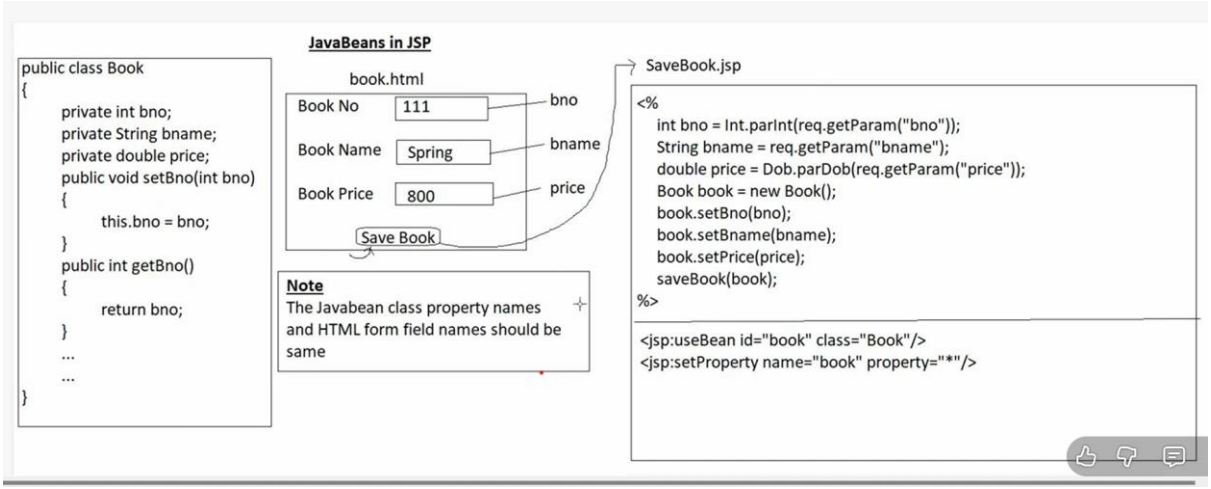
JavaBeans : A JavaBean is a public class with private properties and public setter and getter methods for every property .

Ex :-

```
public class Book {
private int bno;
private double price;


public void setBno(int bno)  {
this.bno  =  bno;
}

public int getBno() {
return bno;
}
// we have to make setter & getter of all variables
. . . .
 . . . .
 }
```



Refer diagram JavaBeans in JSP.png

Example Application :-

- Create a package “mypack” in WebProj
- Create a Javabean class “Book” in mypack package
  Refer program Book.java
- Create a welcome page “book.html” in WebProj
  Refer Program Book.html
- Create a JSP file “SaveBook.jsp” in WebProj
  Refer program saveBook.jsp
- Create a JSP file “DisplayBook.jsp” in WebProj
  Refer program DisplayBook.jsp

- Run the application book.html

Generating Setter and Getters :-

Right click on Book.java -> source -> Generate Setters and Getters -> click Select All ->
Click Select All -> click Generate

JSTL :-

- JSTL stands for JSP Standard Template Library
- JSTL is a set of tags created by Apache company
- In order to use JSTL tags we need to copy "jstl1.2.jar" into WEB-INF/lib folder

Refer programs
- JSTLCore1.jsp
- JSTLCore2.jsp
- JSTLCore3.jsp
-JSTLCore4.jsp

${} => expression language in JSP (EL)

Chapter - 37

JSP Mini Project :-

- Create a HTML file which accepts the customer details like (Customer name, username, password, age and email.
- Create a JSP page which inserts the customer details in Customer Database Table
- **Create a table "Customer" in MySQL**

mysql > create table Customer (custName varchar(10), username varchar(10), password varchar(10), age int(2), email varchar(20));

-Create a Javabean class "Customer" in mypack package of WebProj
Refer program Customer.java
- Create a welcome page "customer.html" in WebProj
Refer program customer.html

- Create a JSP file "SaveCustomer.jsp" in WebProj
Refer program SaveCustomer.jsp

- Create a class "InsertCustomer" in mypack package
Refer program InsertCustomer.java
- Create a JSP file "DisplayCustomer.jsp" in WebProj
Refer Program DisplayCustomer.jsp
- Run the application customer.html

Chapter - 38

Apache Maven Tool :-

- Maven is a powerful project management tool .
- Maven is used to build projects.
- Maven is used to add projects dependency jar files into the project.
- Maven is used to create the right project structure
- Maven is used to build and deploy the project .

Creating a Maven Project in Ecllipse :-

- Click on File -> New -> Maven Project
  Click Next
  In Catalog Select : Internal
  In Artifact Id Select : maven-archetype-quickstart

// we use maven archetype-quickstart (it is for standalone application)

  Click Next
  Group Id : Maven
  Artifact Id : FirstMavenProj (Project Name)
  Package : com.maven
  Click Finish

pom.xml (Project Object Model) :-

- In this pom.xml file we add the project dependency jar files .
- <project> is the root element of pom.xml file

TO add- connector-java-8.0.30.jar files into the project using maven :-

---

Google : MySQL connector java maven
Copy the following dependency code and paste in between <dependencies> element of pom.xml file

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
</dependency>
```

Refer program FirstMavenProject/pom.xml

Framework :-

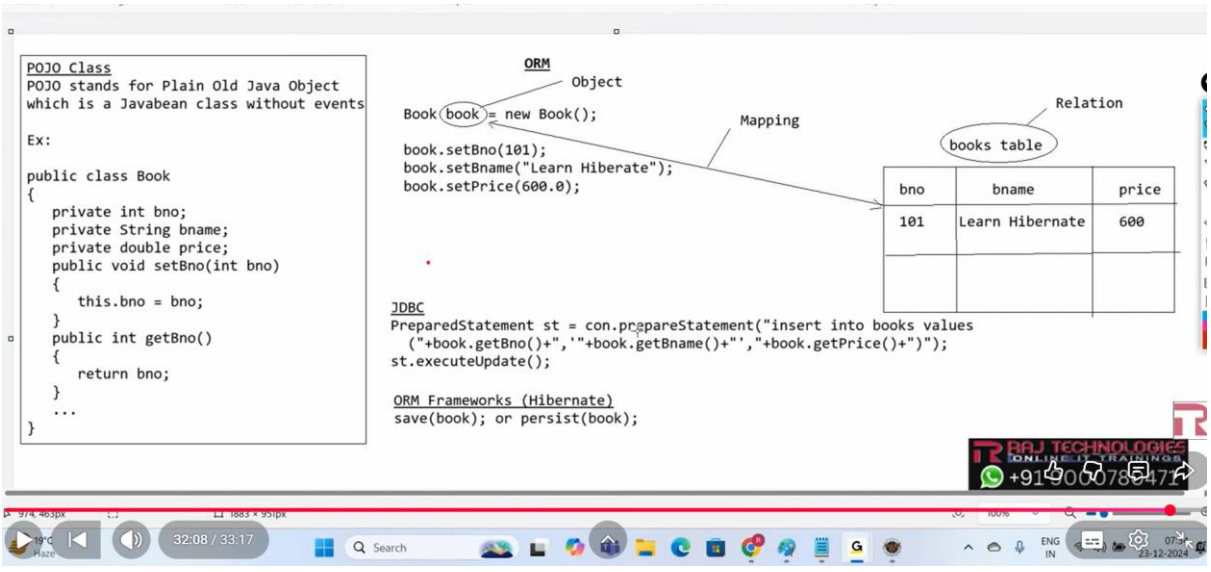
- A framework is a group of prefefined libraries which comes with set of jar files .
- Framework will provide some common logics required for the project development.
- Framework provides reusable components

- Framework will reduce the burden to the developers
  we use two types of Framework :-
    1- Hibernate Framework

    2- Spring Framework
        (i)- Spring Core/ Bean Module
        (ii)- Spring DAO Module
            1- Spring JDBC Module
            2- Spring ORM Module (Hibernate)
            3 – Spring Data JPA Module
        (iii)- Spring MVC Module => used to develop web applications
        (iv)- Spring Boot Module
        (v) - Spring REST Module => used to develop REST APIs – distributed application
    3 – Microservices using Spring Boot – 2class
    4- JUnit Framework – 1class
    5- Angular Framework – 3class

Hibernate Framework :-

- Hibernate is an ORM Framework
- ORM stands for Object Relational Mapping
    Refer Diagram ORM.png
Q – Drawback of JDBC & Advantages of Hibernate



- Hibernate is an ORM solution for Java and it raised as open source persistent framework created by Gavin King.

- Hibernate maps Java classes to database tables and Java class properties with the table column using Hibernate XML mapping file.

Ex : emp.hbm.xml

- Hibernate sits between Java objects and database

  Java Object ←→ Hibernate ←→ Database

  1- Hibernate provides simple APIs for storing and retrieving Java objects directly to and from the database.

  1- Persist() / save() (Deprecated(old method) in Hibernate 6) = > Insert Command

  2- Merge() / update() (deprecated in Hibernate 6) => Update Command

  3- Remove() / delete() (deprecated in Hibernate 6) => Delete command

  4- Get() / list () => select command

- In Hibernate we create Hibernate XML Configuration file in which we configure the database connections like driver class, url , username and password and in this XML file we ever refer to the Hibernate XML mapping files.

  Ex : hibernate.cfg.xml

- If there is a change in database then only need to change is XML file properties

Chapter - 39

Developing a Hibernate Application in Eclipse using Maven :-

1- Create a Maven Project

click on File -> New -> Maven Project

Make check mark to “create a simple Project (skip archtype selection)

Click Next

Group Id : hibernate

Artifact Id : HibernateProj (Project Name)

click Finish

2- Add the following dependencies in pom.xml file of HibernateProj

```
        <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
</dependency>
<!-- Source: https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.6.44.Final</version>
    <type>pom</type>
    <scope>compile</scope>
</dependency>
```

Refer Program pom.xml

3 – Create a package “com.hibernate” in src/main/java folder of HibernateProj

5 – Create a POJO Class “Employee” in com.hibernate package
Refer program Employee.java

6 – Create a Hibernate XML mapping file “emp.hbm.xml” in src/main/resources folder of HibernateProj
(we have created mapping file to map our project class with the database table or project class properties with the table)
Refer Program emp.hbm.xml

7 – Create a Hibernate XML xonfiguration file “hibernate.cfg.xml” in src/main/resources folder
(Configuration file where we have loaded the database connection properties)
Refer program hibernate.cfg.xml

8- Create a utility class “HibernateUtil” in com.hibernate package
Refer program HibernateUtil.java

9- Create a test class “EmployeeInsert” in com.hibernate package
Refer program EmployeeInsert.java (Run)

Important for Interview
Create v/s update values in <property name=”hbm2ddl.auto”>

Create :-

- drop existing table
- create new tables

update :

- use existing tables
- if table does not exist, create new tables

CRUD Operation :-

C => Insert Command => EmployeeInsert.java
R => Select command =>EmployeeList.java
U => update Command =>EmployeeUpdate.java
D =>Delete Command => EmployeeDelete.java

Note :
Create .java files in com.hibernate package of src/main/java folder of HibernateProj

Using Annotation in Hibernate :

- When annotations are used in Hibernate, we need not create Hibernate XML mapping file.
    @Entity
    @Id
    @Table
    @Column

Example Application :-

-Create a POJO Class “Product” in com.hibernate package
    Refer program Product.java
-Update hibernate.cfg.xml file of src/main/resources folder of HibernateProj to add the following mapping class
    <mapping class=”com.hibernate.Product”/>
    Refer program hibernate.cfg.xml

- Create a test class “ProductTest” in com.hibernate package
    Refer program ProductTest.java (Run)

Generating Primary Keys :

@GeneratedValue annotation is used for the primary key generation
    Ex :
        @GeneratedValue(strategy=GenerationType.IDENTITY)

    Refer Programs
        - Product.java (updated)
        -ProductTest.java (updated)

Chapter - 40

Spring Framework :-

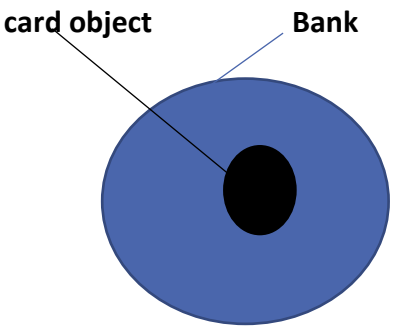
- Spring Framework is an Application Framework
-Spring Framework is used to develop robust web applications and enterprise applications very easily and rapidly .

Dependency Injection :-

- Spring is mostly identified with Dependency Injection which is a flavor of Inversion of Control (IOC)

Ex :-

```
class Card {
object
        . . .
          }
class Bank {
        Card card = new Card();
          }
Bank bank = new Bank();
```



Note :- In the above code , Card Object and bank object are tightly coupled if we make any changes in card we have to make changes in bank also .

Example :-

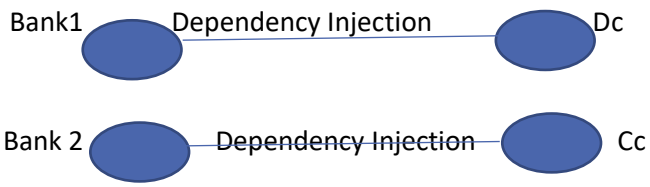
```
interface Card
{
  . . .
}
class DebitCard implements Card
{
  . . .
}
class CreditCard implements Card
{
  . . .
}
class Bank {
        Card card : // property
        Bank(Card card)      // Constructor Injection
              {
             this.card = card;
              }
           void setCard(Card card) {       // Setter Injection
                    this.card  = card;
                          }
                  }
```

Note :- we can inject one object into another only two ways 1- Constructor Injection 2- Seter Injection

```
Card dc = new DebitCard();
Card cc = new CreditCard();
```

Bank bank1 = new Bank(dc); // injecting Debit card reference to Bank1
Bank bank2 = new Bank(cc); // Injecting Credit Card reference to Bank2

Note : Injecting one object into another object is called Dependency Injection.



Note In the above code . Card and Bank Object are loosely coupled. We have to go with loosely coupled applications means any changes to this dependency(Card) we don't have to make changes in bank .

Life Cycle of an Object :-

=> Object Creation -> Object Usage -> Object Destruction
- In java, Object destruction is taken care by Java as Java is automatic garbage collection .
- In Spring, the life cycle of an object is taken care by Spring Framework.
- In Spring, the Object are created using Spring Bean XML configuration file.

Spring Modules :-

- spring Core/Bean Module
- Spring DAO Module
    - Spring JDBC Module
    - Spring ORM Module (Hibernate)
    - Spring Data JPA Module
- Spring MVC Module => used to develop web applications
- Spring Boot Module
- Spring REST Module => Used to Develop REST APIs (distributed applications)

Develop a Spring Application in Ecllipse using Maven :-

1- Create a Maven Project
    Click on Filr -> New -> Maven Project
    Make Click mark to " Create a Simple Project (Skip archetype selection)
    Click Next
    Group Id : spring
    Artifact Id : SpringProj :
    click Finish
2 – Add the Following Depedency in pom.xml file of SpringProj
    <!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-context -->

```
<dependencies>
   <dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-context</artifactId>
   <version>5.1.18.RELEASE</version>
   <scope>compile</scope>
</dependency>
</dependencies>
```

Refer program pom.xml

3- Create a package “com.spring” in src/main/java folder of SpringProj

4 – Create a Spring bean class “Book” in com.spring package

Refer program Book.java

5 – Create a Spring Bean class “Library” in Com.spring package

Refer Program Library.java

6 – Create Spring bean XML configuration file “library.xml” in src/main/resources folder of SpringProj

Refer Program library.xml